OBJECT ORIENTED PROGRAMMING WITH C++
ASSIGNMENT 3 – CO3

Constructors, Destructors and Operator Overloading

Department of Computer Science & Engineering

Submitted by:
Rashmi Naurene
Register Number: 192521357

Under the Guidance of
Jegedeesh K

Academic Year 2025–26 | Total Marks: 50

INDEX

Q1 – Constructors and Constructor Overloading ..... 6 Marks

Q2 – Copy Constructors: Shallow and Deep Copies ..... 7 Marks

Q3 – Dynamic Constructors and Destructors ..... 7 Marks

Q4 – Overloading Arithmetic and Stream Operators ..... 7 Marks

Q5 – Overloading Increment, Comparison and Subscript ..... 6 Marks

Q6 – The Rules of Operator Overloading ..... 7 Marks

Q7 – Design a Complex-Number Class from First Principles ..... 10 Marks

Total: 50 Marks

Q1 – Constructors and Constructor Overloading

Library Book Records

1.1 Class with Overloaded Constructors and Delegation

```
class Book {
    std::string title;
    std::string author;
    int copies;
public:
    // default – empty record to be filled later
    Book() : Book("", "", 0) {}
    // donated book – only title known
    explicit Book(const std::string& t) : Book(t, "Unknown", 1) {}
    // full details
    Book(const std::string& t, const std::string& a, int c)
        : title(t), author(a), copies(c < 0 ? 0 : c) {}
    void print() const;
};
```

Every constructor uses a member initializer list. The two simpler constructors delegate to the most general one, so the validation of copies exists in only one place. A negative number of copies is forced to zero; no Book can be created with an invalid count.

1.2 When the Compiler Supplies a Default Constructor

If the programmer writes no constructor at all, the compiler generates a public default constructor. As soon as any user-declared constructor exists, the compiler-generated default constructor is suppressed. That is why removing the default constructor above makes Book books[10]; a compile-time error.

1.3 Initializer Lists vs Assignment in the Body

Members are initialised before the constructor body runs. An initializer list constructs them directly; assignment inside the body default-constructs them first and then assigns, which is wasteful for types with non-trivial constructors and impossible for const or reference members.

1.4 Declaration Order, not List Order

Members are always initialised in the order of their declaration inside the class, regardless of the order written in the initializer list. Compiling with -Wreorder warns when the two orders differ.

1.5 explicit Constructors

A single-argument constructor should normally be explicit so that it cannot act as an implicit conversion. Without explicit, Book b = std::string("Title"); compiles; with explicit it is rejected. Use explicit whenever the conversion would be surprising.

1.6 Alternative Concept

One constructor with default arguments (Book(std::string t="", std::string a="", int c=0)) also works and still centralises the copies check. Object-creation time and space remain O(1). Overloading + delegation makes each construction intent clearer at the call site; default arguments make the interface shorter.

Q2 – Copy Constructors: Shallow and Deep Copies

Hospital Prescription Records

2.1 The Shallow-Copy Problem

Without a user-defined copy constructor the compiler generates a member-wise (shallow) copy. Both Prescription objects then hold the same pointer. When the first object is destroyed it deletes the array; the second destructor deletes the same address again → undefined behaviour (double-free), often reported by AddressSanitizer as "attempting free on address which was not malloc()-ed" or a heap-use-after-free.

2.2 Complete the Code – Deep Copy (Rule of Three)

```
// TODO 1 – Copy constructor
Prescription(const Prescription& other)
    : n(other.n), medicines(new std::string[other.n]) {
    for (int i = 0; i < n; ++i) medicines[i] = other.medicines[i];
}

// TODO 2 – Copy assignment
Prescription& operator=(const Prescription& other) {
    if (this == &other) return *this;          // self-assignment guard
    std::string* neu = new std::string[other.n];
    for (int i = 0; i < other.n; ++i) neu[i] = other.medicines[i];
    delete[] medicines;
    medicines = neu;
    n = other.n;
    return *this;
}

// TODO 3 – Destructor
~Prescription() { delete[] medicines; }
```

Expected output confirmed: Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS. The three objects are independent.

2.3 Rule of Three and When the Copy Constructor Is Called

If a class manages a resource, it almost always needs a destructor, a copy constructor and a copy-assignment operator together (the Rule of Three). The copy constructor is invoked when:

- an object is initialised from another object of the same type,
- an object is passed by value to a function,
- an object is returned by value from a function (subject to NRVO).

2.4 Experimental Notes

Passing by value copies once; passing by const reference copies zero times. Copying 100 000 medicine names is O(n) time and O(n) extra space. Self-assignment (p = p) is protected by the identity test. After deleting the user-defined copy constructor the sanitizer again reports a double-free.

2.5 Alternative Concept – Rule of Zero

Storing the names in a std::vector<std::string> lets the compiler-generated special members perform a correct deep copy automatically. Time and space remain O(n); the programmer writes no destructor, copy constructor or assignment operator.

Q3 – Dynamic Constructors and Destructors

Auditorium Seat Booking

3.1 Dynamic Constructor and Destructor

```
class Event {
   int rows, seats;
   bool** map;              // map[r][s]
public:
   Event(int r, int s) : rows(r), seats(s), map(nullptr) {
      if (rows <= 0 || seats <= 0) return;
      map = new bool*[rows];
      for (int i = 0; i < rows; ++i)
         map[i] = new bool[seats]();   // value-initialised false
   }
   ~Event() {
      if (!map) return;
      for (int i = 0; i < rows; ++i) delete[] map[i];
      delete[] map;
      std::cout << "Event closed (" << rows << "x" << seats << ")\n";
   }
   // book / cancel omitted for brevity
};
```

3.2 Construction / Destruction Order

Local automatic objects are destroyed in reverse order of construction when their block ends. A heap object lives until delete is called. If delete is omitted the memory leaks; if delete (instead of delete[]) is used on an array the behaviour is undefined. A destructor takes no arguments and cannot be overloaded because the compiler must know exactly which function to call at the end of an object's lifetime.

3.3 Predict the Output (Logger)

My predicted output:

```
+A +B -B copyAc use -Ac +P -P end -A
```

Actual output matches. a is created first and destroyed last. b is created and destroyed inside the inner block. use(a) passes by value, so a copy is made and destroyed at the end of the full-expression. The heap object P is created with new and destroyed by delete.

3.4 RAII Conclusion

Tying resource release to the destructor guarantees that every exit path (normal return, exception, early return) still frees memory. This is the essence of RAII and the main reason C++ programs can be exception-safe without explicit finally blocks.

3.5 Alternative Concept

A single contiguous allocation (bool* map = new bool[rows*seats]) needs only one new and one delete[], reducing allocator traffic. std::vector<std::vector<char>> gives the same logical layout with automatic cleanup. Access remains O(1); the pointer-to-pointer form uses extra space for the row pointers.

Q4 – Overloading Arithmetic and Stream Operators

Money Arithmetic for a Billing Counter

4.1 Complete the Code

```
Money operator+(const Money& other) const {
   return Money(0, paise + other.paise);   // constructor normalises
}
bool operator==(const Money& other) const {
   return paise == other.paise;
}
friend std::ostream& operator<<(std::ostream& os, const Money& m) {
   long long r = m.paise / 100;
   int p = static  cast<int>(m.paise % 100);
   if (p < 0) p = -p;
   os << "Rs " << r << '.' << std::setw(2) << std::setfill('0') << p;
   return os;
}
```

Expected output confirmed: Rs 200.00 / true. Carry test: Money(0,75) + Money(0,50) yields Rs 1.25.

4.2 Design Decisions

- + returns a new object because the mathematical sum is a distinct value; += returns *this so that chaining is possible.
- << must be a non-member (usually a friend) because its left operand is an ostream, not a Money.
- Binary operators that treat both operands symmetrically are often friends; mutating operators such as += are naturally members.
- The compiler translates a + b into either a.operator+(b) or operator+(a,b) according to the declarations in scope.

4.3 Engineering Decision – Which Operators Make Sense

Addition, subtraction and comparison are meaningful for money. Multiplication of two Money amounts is not (Rs × Rs has no monetary meaning); multiplication by a scalar (quantity) is. Division by a scalar is useful for unit prices; division of two Money values yields a pure number. The class therefore overloads only the operators that preserve the abstraction.

4.4 Alternative Concept

Non-member friends allow 100 + m as well as m + 100 through implicit conversion of the left-hand operand. Named functions add/equals are clearer to some readers but lose the algebraic readability of operator notation. All forms are O(1) time and space.

Q5 – Overloading Increment, Comparison and Subscript

Music Playlist and Track Timer

5.1 Prefix vs Postfix ++

```
Time& operator++() {          // prefix: increment, return new value
   if (++sec == 60) { sec = 0; if (++min == 60) { min = 0; ++hour; } }
   return *this;
}
Time operator++(int) {        // postfix: return old value, then increment
   Time old = *this;
   ++(*this);
   return old;
}
```

The dummy int parameter tells the compiler which version is intended. Postfix must return a copy of the old state, making it more expensive for class types.

5.2 Predict the Output (Counter)

My predicted output: 5 7 7

Actual output: 5 7 7. a receives the old value (5) from postfix; b receives the already-incremented value (7) from prefix; c ends at 7. Only the postfix version made a copy.

5.3 operator[] – const and non-const

A non-const overload returns a reference so the caller can modify the element. A const overload returns a const reference (or a copy) so that a const Playlist can still be read. Out-of-range indices should throw std::out_of_range or otherwise signal the error; silent wrapping hides bugs.

5.4 Why Prefer ++i for Class Types

Prefix avoids the temporary copy that postfix must create. After inlining and optimisation the difference may disappear for trivial types, but the habit of writing ++i remains good practice for user-defined iterators and counters.

5.5 Alternative Concept

An at(i) that throws is safer for public APIs; an unchecked [] is fastest when the index is already known to be valid. The extra comparison of a checked [] is O(1) and measurable only in tight micro-benchmarks (tens of nanoseconds per call).

Q6 – The Rules of Operator Overloading

Simple Text Class for an SMS Gateway

6.1 Operators That Cannot Be Overloaded

:: . .* ?: sizeof typeid alignof and the preprocessor symbols cannot be overloaded. They are fundamental to the language's name-lookup and parsing rules; changing their meaning would make programs unreadable.

6.2 Core Rules

- Precedence, associativity and arity are fixed; only the implementation can be supplied.
- At least one operand must be of a user-defined type (you cannot redefine int + int).
- operator= , operator[] , operator() and operator-> must be non-static member functions.
- Symmetric binary operators are often implemented as friends so that conversions apply to both sides.

6.3 Text Class Outline (Rule of Three + Operators)

```
class Text {
   char* data; std::size  t len;
public:
   Text(const char* s = "");
   Text(const Text& o);                // deep copy
   Text& operator=(const Text& o);     // deep copy, self-assignment safe
   ~Text() { delete[] data; }
   Text operator+(const Text& o) const; // concatenation
   bool operator==(const Text& o) const;
   bool operator!=(const Text& o) const { return !(*this == o); }
   friend std::ostream& operator<<(std::ostream&, const Text&);
};
```

6.4 Experimental Observations

Evaluating a + b + c creates at least one temporary (sometimes two, depending on NRVO). AddressSanitizer reported no leaks after the destructor was written correctly. Self-assignment is guarded. For 100 000 concatenations std::string is both faster and safer; the hand-written Text class nevertheless demonstrates ownership mechanics clearly.

6.5 Conclusion

Writing Text teaches the Rule of Three, deep-copy discipline and the constraints of operator overloading. In production code one would simply use std::string unless a severely constrained environment forbade the standard library.

6.6 Alternative Concept

Implementing + by taking the left operand by value and using += (Text operator+(Text a, const Text& b){ a += b; return a; }) enables move semantics automatically. Character-copying cost for a+b+c remains proportional to the total length; the number of allocations can be reduced by reserving capacity.

Q7 – Design a Complex-Number Class from First Principles

AC Circuit Impedance Calculator

Stage 1 – Interpret the Problem

Data required: real part and imaginary part (both double). Operations required: addition and subtraction (series), multiplication and division (parallel combination), equality, magnitude, and formatted output. Constructors needed: default (0+0j), from one double (real + 0j), from two doubles. Measurable requirements:

- series(Z1,Z2) equals the hand-calculated sum,
- parallel formula (Z1*Z2)/(Z1+Z2) matches a reference value within 1e-9,
- division by zero impedance yields a defined error state rather than a crash.

Stage 2 – Design

```
class Complex {
   double re, im;
public:
   Complex(double r = 0, double i = 0) : re(r), im(i) {}
   Complex operator+(const Complex& o) const { return {re+o.re, im+o.im}; }
   Complex operator-(const Complex& o) const { return {re-o.re, im-o.im}; }
   Complex operator*(const Complex& o) const {
      return {re*o.re - im*o.im, re*o.im + im*o.re};
   }
   Complex operator/(const Complex& o) const {
      double d = o.re*o.re + o.im*o.im;
      if (d == 0.0) throw std::runtime_error("division by zero impedance");
      return {(re*o.re + im*o.im)/d, (im*o.re - re*o.im)/d};
   }
   bool operator==(const Complex& o) const {
      return re == o.re && im == o.im;
   }
   double mag() const { return std::sqrt(re*re + im*im); }
   friend std::ostream& operator<<(std::ostream& os, const Complex& z) {
      os << z.re << (z.im >= 0 ? " + " : " - ") << std::abs(z.im) << "j";
      return os;
   }
};
```

Arithmetic operators are members (left operand is already Complex). << is a friend so it can be called with an ostream on the left.

Stage 3 – Implement and Verify

Example series R–L circuit: Z = 30 + j40. Hand calculation magnitude = 50. Program prints 30 + 40j and mag() = 50. Parallel combination of 30+j40 and 0−j30 yields a result that matches the algebraic evaluation to machine precision. Division by Complex(0,0) throws, satisfying the safety requirement.

Stage 4 – Defend the Design

std::complex<double> already supplies the same operators and is thoroughly tested; a production program should prefer it. Writing the class nevertheless demonstrates mastery of constructors, operator overloading and error handling. Each constructor is justified by a distinct creation scenario; each operator corresponds to a required circuit law. Remaining limitation: the class does not yet provide polar-form constructors or

mixed operations with plain doubles on the left-hand side (those can be added as non-member overloads if needed).

Alternative Concept

Using std::complex<double> reduces the implementation to a few typedefs and helper functions. Time and space per operation are identical (two doubles, O(1)). The standard class does not throw on division by zero; the caller must test the denominator explicitly. The educational value of the hand-written class is higher; the engineering value of the standard class is higher.